

Improving choice model specification using reinforcement learning

Gabriel Nova^{*1}, Sander van Cranenburgh¹, Stephane Hess²

¹Transport and Logistics group, Delft University of Technology, The Netherlands

²Choice Modelling Centre - Institute for Transport Studies, University of Leeds, England

Discrete choice modelling is a theory-driven modelling framework for understanding and forecasting choice behaviour. To obtain behavioural insights, modellers test several competing model specifications in their attempts to discover the “true” data generation process. This trial-and-error process requires expertise, is time-consuming, and relies on subjective theoretical assumptions. Although metaheuristics have been proposed to assist choice modellers, they treat model specification as a classic optimisation problem, relying on static strategies, applying predefined rules, and neglecting outcomes from previous estimated models. As a result, current metaheuristics struggle to prioritise promising search regions, adapt exploration dynamically, and transfer knowledge to other modelling tasks. To address these limitations, we introduce a deep reinforcement learning-based framework where an “agent” specifies models by estimating them and receiving rewards based on goodness-of-fit and parsimony. Results demonstrate the agent dynamically adapts its strategies to identify promising specifications across data generation processes, showing robustness and potential transferability, without prior domain knowledge.

Keywords: Choice modelling, DQN, Deep Reinforcement learning, Custom environment

1. INTRODUCTION

Discrete choice analysis is widely employed in travel behaviour research to understand travellers' preferences and predict travel demand. Extracting meaningful insights from such models requires the careful selection of an appropriate specification, a process that is often time-consuming due to the numerous modelling decisions involved (van Cranenburgh et al., 2022). For example, consider a researcher has collected mode choice data containing attributes such as travel time, travel cost, and service frequency. Faced with these complex data, the researcher typically examines multiple competing model specifications. This involves choosing an appropriate model family, such as the Multinomial Logit, Nested Logit, or Mixed Logit framework, identifying relevant attributes, determining functional forms, and addressing both observed and unobserved sources of heterogeneity. This process is often considered an art since it combines the subjective judgments and expert knowledge of researcher (Nova et al., 2024).

Recent metaheuristics have been developed to assist travel behaviour researchers in the model specification phase. Paz et al., (2019) used Simulated Annealing, which explores local neighbourhoods of specifications to improve goodness-of-fit. Rodrigues et al., (2020) introduced a Bayesian framework that uses automatic relevance determination to select optimal specifications. Ortelli et al., (2021) considered a Variant Neighbourhood Search that iteratively applies operators to enhance neighborhoods model performance. Beeramoole et al., (2023) proposed a bi-level constrained optimization framework that integrates prior assumptions into the search process to align specification with theoretical expectations. Finally, Haj-Yahai et al., (2024) used Grammatical Evolution to evolve specifications through genetic operators guided by a novel specific grammar.

Despite their applicability to several problems to identify promising candidates (Bianchi et al., 2008), current metaheuristics fail to capture the dynamic and learning nature of the model specification process. They rely on static strategies and predefined rules, which restrict their ability to incorporate knowledge gained during exploration process, as human analysts do. They also lack mechanisms to store and exploit previous experiences, leading to potential inefficiencies such as re-exploration in non-promising areas. In addition, most metaheuristics treat model specification as single task, which requires a new search process for each dataset or objective function, making it difficult to transfer knowledge across tasks.

In this paper, we propose a novel data-driven framework that leverages Reinforcement Learning (RL) to automate the search process for suitable model specifications. RL is a machine learning paradigm where agents interact with an environment to learn a sequence of actions that maximises a cumulative reward signal (Sutton and Barto, 2018). The agent observes the state of the environment, takes actions, and receives feedback in the form of a reward, which reflects the quality of its decision relative to its objective (Hayashi et al., 2019). Research demonstrates that RL algorithms can learn from scratch to optimise workflows, develop new strategies, or solve complex problems more efficiently than human experts. For example, AlphaGo showed the promising power of Deep RL by mastering the game of Go through self-play and without prior expert input (Mnih et al., 2015). Similarly, Zou and Li (2018) applied Deep RL to model experts driving patterns and optimise vehicle control. Beyond these applications, RL has also been applied to design machine learning models. For instance, Baker et al. (2016) developed a meta-modelling algorithm that uses RL to generate high-performing convolutional neural networks. These examples evidence RL's adaptability and potential for automating the search process in choice modelling.

The remainder of the paper is organised as follows: Section 2 details the proposed framework and training procedure, Section 3 explains the study cases, Section 4 presents preliminary results, and Section 5 provides a conclusion and expected results.

2. METHOD

This study introduces a novel framework based on RL to automate the search process of model specifications. Below, we provide an overview of relevant RL concepts, describe the proposed framework, and explain its interaction with the modelling space environment.

2.1. Reinforcement learning paradigm

Reinforcement learning is a branch of machine learning wherein agents learn to make decisions by interacting with an environment and receiving feedback in the form of rewards (Sutton and Barto, 2018). Specifically, the agent’s objective is to learn an optimal policy (π_θ^*) that maximises the expected cumulative reward. To achieve this, the agent balances exploration (trying new actions) and exploitation (choosing actions known to yield higher rewards) (Burnetas and Katehakis, 1997).

In this paradigm, the agent-environment interaction is modelled as a Markov Decision Process, represented by a tuple (S, A, P, R) . In this notation, S determines the State space, A defines the discrete Action space, $P(s_{e+1}|s_e, a_e)$ represents the probability of transitioning from state s_e to state s_{e+1} given action a_e , $R(s_e, a_e)$ defines the reward function. Thus, at each episode e , the agent observes the current state s_e , selects an action a_e based on a policy $\pi_\theta(a_e|s_e)$ parameterised by policy weights (θ), receives a reward r_e , and transitions to a new state s_{e+1} , aiming to maximise the cumulative discounted rewards over an infinite horizon using a factor (γ).

Traditional approaches have relied on value-based methods, such as Q-learning (Watkins & Dayan, 1992), and policy-based methods like Policy Gradient (Sutton et al., 1999) for learning the optimal policy. On the one hand, value-based methods focus on learning the optimal value function and deliver the optimal policy indirectly by choosing actions that maximize the action-value function $Q(s, a)$, as shown in Eq. (1). On the other hand, policy-based methods directly parameterise the policy $\pi_\theta(a|s)$ and update parameters θ by performing gradient ascent to maximize the expected cumulative rewards, as shown in Eq. (2) (Schulman et al., 2016).

$$\pi_\theta^*(s) = \underset{a}{\operatorname{argmax}} Q(s, a; \theta) \quad (1)$$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}[\log(\pi_{\theta(a|s)}) \cdot Q(s, a; \theta)] \quad (2)$$

These traditional methods become infeasible in environments with high-dimensional, continuous or dynamic state-action spaces, since they require maintaining a Q-value for all possible state-action pairs (Plaat, 2022). Deep RL address these limitations by using Deep Neuronal Networks to approximate the Q-value functions or policies (Mnih et al., 2016). The pioneering Deep Q-Network (DQN) not only uses a neural network to replace the Q-value function, but also employs experience replay and target networks to stabilise the training process (Mnih et al., 2015). While experience replay mitigates temporal correlations in action-value updates by randomly sampling past interactions from a replay buffer (Lin, 1992), the target network stabilises Q-value estimation by providing temporal target Q-values. This latest approach reduces discrepancies and prevents oscillatory divergence during training (Tsitsiklis et al., 1997).

2.2. RL framework for automating model specification

The proposed RL framework uses a DQN agent to automate the process of specifying Multinomial Logit models by sequentially interacting with a custom environment to maximise cumulative rewards. Rather than mimicking the human modeller’s decisions, the agent independently acts on a specification to select variables, apply transformations, incorporate interactions, and decide when to estimate the model. Then, the environment estimates the agent’s specification and

provides the modelling outcomes. Several episodes enable the agent to explore the model space, learn to take actions and improve specifications based on a reward signal.

To illustrate this process, consider a transport mode choice scenario involving different alternatives and attributes: travel time (TT), travel cost (TC), headway (HW), and number of changes (CH). In this context, the agent’s goal is to specify a Multinomial Logit model. At the start of an episode, the agent begins with an empty state ($S_e = \{ \}$), which represents an unspecified model. As the episode progresses, the agent iteratively takes actions, such as adding attributes, modifying transformations, or terminating the process, as shown in Figure 1.

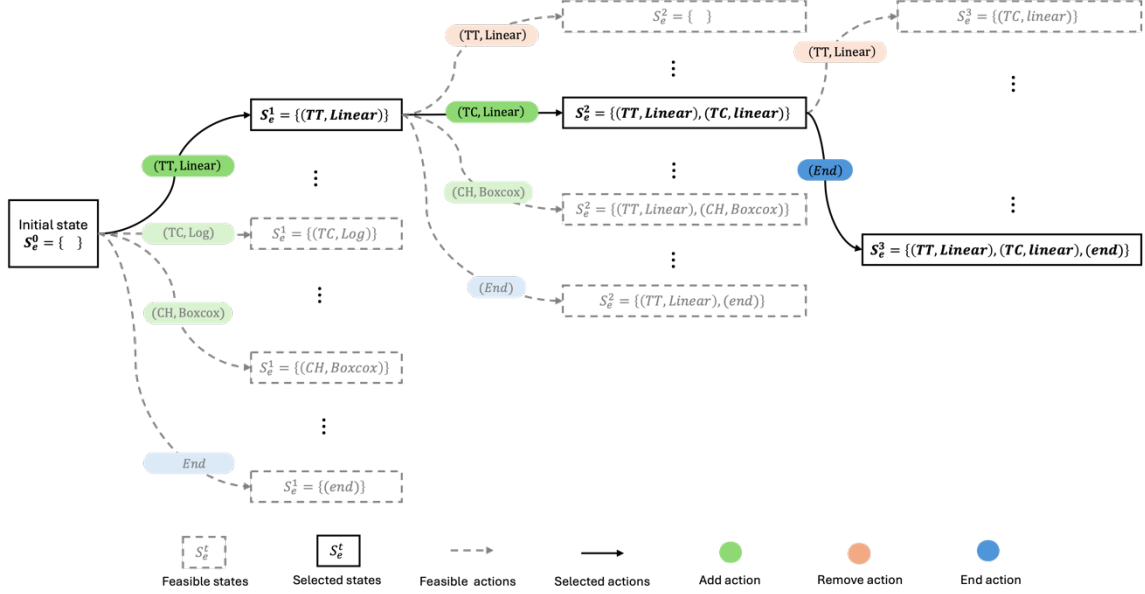


Figure 1: Agent’s sequential decision-making process. Each node represents an internal state; each segmented arrow represents a feasible action, and each solid arrow represents an action the agent took.

At each step, the internal state (S_e^l) is encoded as a one-hot vector, where each element represents an (attribute, transformation) pair (Section 2.2.1). This vector is passed through the input layer of the DQN agent, as depicted in Figure 2. The policy network, which includes fully connected hidden layers, processes this encoded state and learns abstract representations. Then, the output layer provides Q-values for all valid actions, which the agent uses them to select an action using an ϵ -greedy strategy, balancing exploration and exploitation. Upon termination, the environment estimates the proposed MNL model and returns the modelling outcomes. These outcomes might be used to compute the reward signal that guides the agent’s learning process. The transitions between internal states, selected actions, and rewards are stored as tuples (s, a, r, s') in a replay buffer. During training, the agent samples minibatches, a small subset of stored transitions, to update the policy network parameters.

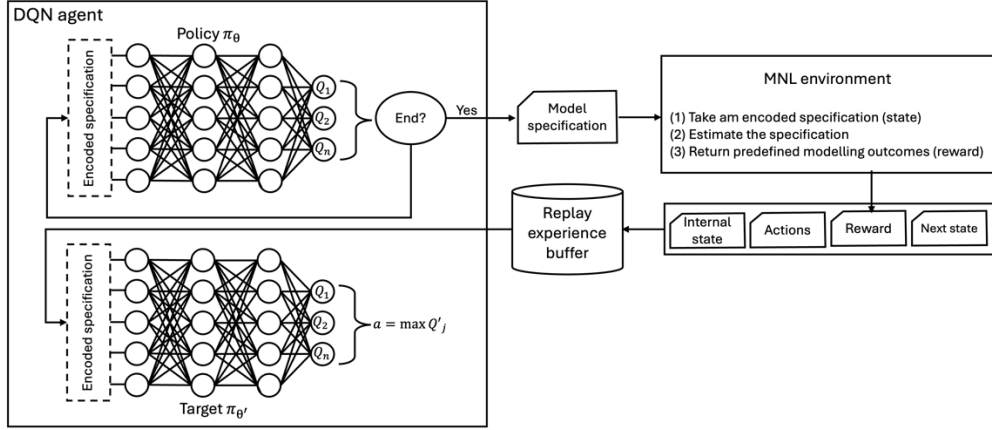


Figure 2: DQN framework for assisting choice model specification.

2.2.1. The state space

We represent model specifications as states encoded as lists of attribute-transformation pairs. This concise representation captures only selected attributes, allowing flexibility to describe any model within the modelling space. Formally, a state S_e^l represents the attribute-transformation considered up to the action l in an episode e :

$$S_e = \{ (k_1, t_1), (k_2, t_2), \dots, (k_l, t_l) \} \quad (3)$$

Future extensions could include tuples that incorporate elements such as model family (M), alternative-specific coefficients (g_i), attributes treated as categorical (c_i), inclusion of interactions (i_i), parameter distributions (d_i), among other as in Eq. (5)

$$S_e = \{ (M, k_1, t_1, g_i, c_i, i_1, d_1), \dots, (M, k_l, t_l, g_l, c_l, i_l, d_l) \} \quad (4)$$

2.2.2. The action space

The action space defines the operations that the agent can perform to modify a specification. The set of valid actions is dynamically generated based on the current state to ensure feasible modifications and avoid loops. At each step l , the agent can select one of the following actions:

- Add a new attribute-transformation pair to expand the model.
- Remove an existing attribute-transformation pair to simplify the model.
- Change the transformation of an existing attribute in the model.
- Terminate the model specification for estimation.

As with the state space, future work could expand the range of actions to include more operations, such as defining interactions, specifying alternative-specific coefficients, adjusting parameter distributions, or incorporating new structural elements.

2.2.3. Reward function

The reward function guides the agent's learning by providing incentives for intermediate actions that improve long-term modelling outcomes. Following the approach of Zhong et al. (2018), the environment provides feedback on intermediate steps by distributing the total reward R_e equally across all intermediate steps in an episode. Thus, at each intermediate step $l \in L$, the agent receives a reward $r_e^l = R_e/L$.

2.2.4. The DQN agent

Our RL agent is an adaptation of the DQN framework to approximate state-action values $Q(s, a, \theta)$, which enable to calculate the cumulative reward of taking an action a in state s under a policy $\pi_\theta = P(a|s)$. Using a fully connected neural network, the agent selects actions with the highest Q-values to transition to new states by interacting with the MNL environment, as shown Eq. (9). To stabilize training, we have also incorporated an experience replay buffer that stores interactions as tuples (s, a, r, s') , and a target network ($\pi_{\theta'}$) that reduce large update oscillations by holding Q-values fixed before periodic synchronisation with the policy network.

$$Q(s, a; \theta) = \max_{\pi_\theta} \mathbb{E} \left[\sum_{l=0}^t \gamma^l \cdot R(s_l | a_l) \middle| s_t = s, a_t = a, \pi_\theta \right] \quad (5)$$

2.2.5. Learning training process

The DQN agent is trained using an ϵ -greedy exploration strategy with minibatches from the replay buffer for Q-learning updates (Mnih et al., 2015). The ϵ parameter controls exploration and exploitation and decreases from 1.0 to 0.1 on a predefined schedule. Initially, the agent explores the modelling space by performing random actions and storing interactions in the replay buffer. Minibatches sampled from the buffer are then used to update the policy and guide the agent's actions.

2.2.6. The MNL environment

The MNL environment offers a controlled setting for specifying Multinomial Logit models using a predefined reward signal. It can be initialised with empirical or generated choice datasets, which remain fixed across episodes. During each interaction, the environment processes encoded specification, estimates it, and returns modelling outcomes, including the log-likelihood, the (adj.) rho-square, estimate values, standard errors, t-tests, AIC, BIC, gradient norm, a flag indicating successful estimation, among others. Built with Apollo functions (Hess and Palma, 2019), this environment meets the Open AI Gym requirements (Brockman et al., 2016) that ensure compatibility with RL methods. While the reward function is customizable, this study uses $\bar{\rho}^2$ for converged models. To align learning with practical expectations, behavioural constraints, such as penalising models with unexpected sensitivities for cost and travel time, can be included in the reward signal.

3. MONTE CARLO EXPERIMENT

Our DQN agent's main task in this novel application is to specify MNL models by considering generic taste parameters while allowing for attribute transformations. The agent is trained considering $\bar{\rho}^2$ as the reward signal for successfully converged models. For non-convergent models, the agent receives a reward equivalent to the null model, where all parameters are set to zero.

In this first experiment, we simulated a single dataset incorporating multiple data generation processes, where the same functional form and parameter values are shared across alternatives. This allows us to know the "truth" and to thus establish how well the RL algorithm performs. For each experiment, a 'true' data generation process is defined, which may include alternative-specific constants, attributes and transformations (logarithmic or Box-Cox), along with their respective coefficients (randomly sampled between -2 and 2), to simulate the choices.

4. RESULTS

Figure 3 illustrates the DQN’s learning process across multiple data generation processes (s), shown as the rolling mean of rewards over batch sizes. Each experiment ran for 8,000 episodes and detailed graphs are provided in the Appendix. The learning curves show an overall upward trend in reward values, which indicates the agent’s increasing capability to identify model specifications that improve goodness-of-fit and parsimony. Fluctuations in the slopes reflect the difference across the data generation processes, with simpler scenarios enabling rapid convergence and more complex ones requiring gradual learning. For example, the lower red line corresponds to a simpler data generation process with fewer influential attributes. The red vertical lines mark reductions in the exploration parameter, which coincides with shifts to greater exploitation of learned policies. These results demonstrate the effectiveness of the exploration-exploitation framework, which enables the agent to refine its policy and adapt to the specific characteristics of each scenario. The upward trend suggests that running beyond 8,000 episodes could uncover better specifications, as the agent continues to build on prior experience. Developing a formal stopping criterion and incorporating tests, such as the log-likelihood ratio test, to compare models are promising areas for future research.

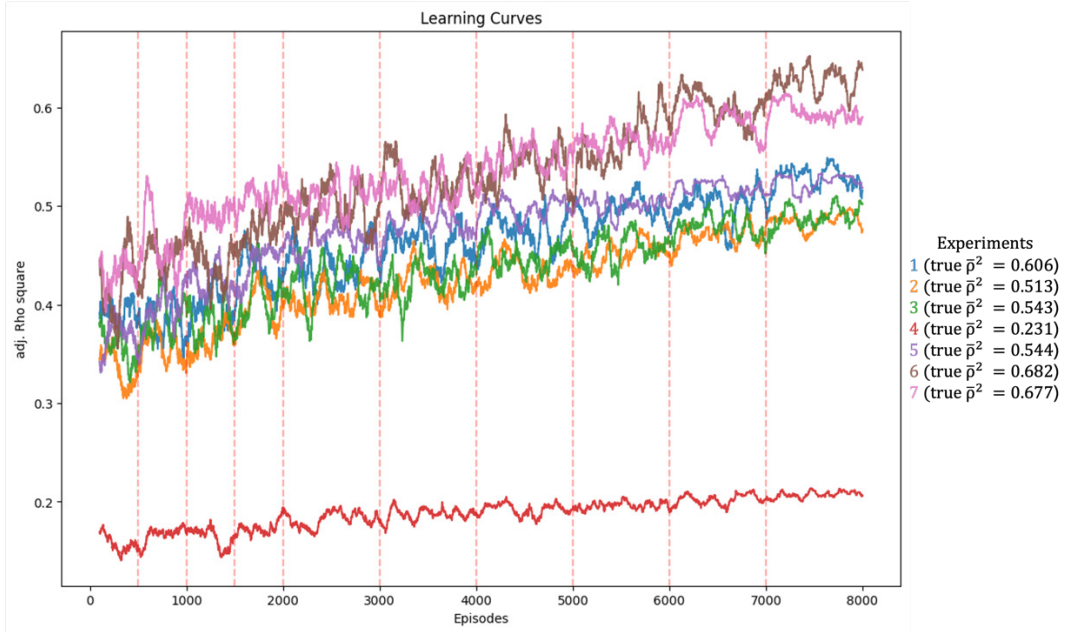


Figure 3: Learning curves of the DQN agent across different data generation processes.

5. CONCLUSIONS

This study demonstrates the potential of reinforcement learning to automate the search process of model specification, which address some limitations of traditional metaheuristics. By allowing the agent to explore the model space without relying on static strategies and refine specifications based on its own experiences, the agent learns to propose specifications that balance goodness-of-fit and parsimony. Moreover, preliminary results show the agent’s ability to adapt to diverse data generation processes and deliver meaningful specifications.

Future effort will focus on enhancing the robustness and generalization of this framework by incorporating statistical tests reduce overfitting and penalties for behavioral constraint violations. Furthermore, we plan to compare our approach with state-of-the-art metaheuristic approaches. At the conference, we will present the results of applying this framework to the Swissmetro dataset, including more complex model structures and alternative model families.

REFERENCES

- Abbeel P, Coates A, Ng AY. Autonomous Helicopter Aerobatics through Apprenticeship Learning. *The International Journal of Robotics Research*. 2010;29(13):1608-1639. doi:10.1177/0278364910371999
- Beeramoole, P. B., Kelly, R., Haque, M. M., Scott, P., Pinz, A., & Paz, A. (2023, November). A metaheuristic-based extensive hypothesis testing for specification of latent class models. In *Australasian Transport Research Forum (ATRF)*, 44th, 2023, Perth, Western Australia, Australia.
- Bianchi, L., Dorigo, M., Gambardella, L. M., & Gutjahr, W. J. (2009). A survey on metaheuristics for stochastic combinatorial optimization. *Natural Computing*, 8, 239-287.
- Burnetas, A. N., & Katehakis, M. N. (1997). Optimal adaptive policies for Markov decision processes. *Mathematics of Operations Research*, 22(1), 222–255.
- Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4), 293–321.
- Haj-Yahia, S., Mansour, O., Toledo, T. (2024). Grammar-Driven Utility Specification: A Grammatical Evolution Approach for Discrete Choice Models. *17th International Conference on Travel Behaviour Research*
- Hayashi, Y., Hasegawa, T., & Ishii, S. (2019). Deep reinforcement learning: A brief survey. *Artificial Life and Robotics*, 24(2), 201–206.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Mnih, V. (2016). Asynchronous Methods for Deep Reinforcement Learning. *arXiv preprint arXiv:1602.01783*.
- Nova, G., van Cranenburgh, S., & Hess, S. Understanding the Decision-Making Process of Choice Modellers. Available at SSRN 5041920.
- Ortelli, N., Hillel, T., Pereira, F. C., de Lapparent, M., & Bierlaire, M. (2021). Assisted specification of discrete choice models. *Journal of choice modelling*, 39, 100285.
- Paz, A., Arteaga, C., & Cobos, C. (2019). Specification of mixed logit models assisted by an optimization framework. *Journal of choice modelling*, 30, 50-60.
- Plaa, A. (2022). Deep reinforcement learning (Vol. 10, pp. 978-981). Singapore: Springer.
- Rodrigues, F., Ortelli, N., Bierlaire, M., & Pereira, F. C. (2020). Bayesian automatic relevance determination for utility function specification in discrete choice models. *IEEE Transactions on Intelligent Transportation Systems*, 23(4), 3126-3136.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems* (pp. 1057–1063).

R.S. Sutton, D. Precup, S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning, *Artif. Intell.* 112 (1–2) (2000) 181–211.

Tsitsiklis, J. N., & Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5), 674–690.

Van Cranenburgh, S., Wang, S., Vij, A., Pereira, F., & Walker, J. (2022). Choice modelling in the age of machine learning-discussion paper. *Journal of choice modelling*, 42, 100340.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.

Zhong, S., Zhang, Y., & Wang, J. (2018). Deep reinforcement learning in parameterized action space. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence* (pp. 5744–5751).

Zou, Q., & Li, H. (2018). Inverse Reinforcement Learning via Neural Network in Driver Behavior Modeling. In *IEEE Intelligent Vehicles Symposium, Proceedings* (number 61673084, pp. 1245–1250).

APENDIX

